Repository-Aware Metamorphic Relation Generation for Augmented Reality Applications using Large Language Models

Dibyendu Brinto Bose
Virginia Tech
Blacksburg, VA, USA
brintodibyendu@vt.edu

Jiawei Qin
Virginia Tech
Blacksburg, VA, USA
jiaweiq@vt.edu

Chris Brown
Virginia Tech
Blacksburg, VA, USA
dcbrown@vt.edu

Abstract

Metamorphic Testing (MT) provides a promising approach for testing software without defined test oracles by specifying expected relations between inputs and outputs, instead of relying on exact outputs. For example, testing Augmented Reality (AR) applications is challenging due to dynamic interactions between virtual content, physical environments, and code, which make traditional test oracles difficult to define. However, formulating metamorphic relations (MRs) is time-consuming and burdensome. We introduce a context-aware pipeline that generates and refines MRs using repository-level context and reasoning orchestration, evaluated on a dataset of 142 mobile AR system repositories. Across three context configurations generating 14,916 candidate MRs, hierarchical context yielded the broadest coverage (7,004 MRs across 142 repositories and 5,167 class–method pairs) and lower redundancy. An agentic deliberation process then reconciled conflicting candidates—observed in 79.0% of cases—reducing duplication and selecting context-aware relations in 88.2% of outcomes. A manual oracle study shows refined relations ($n = 141$) are both logically valid and sufficiently concrete to be directly translated into test assertions, and a preliminary case study reveals converting generated MRs ($n = 5$) into executable tests can detect non-equivalent mutations in real-world code. Overall, our results show that combining repository-aware MR generation with reasoning-based refinement enables scalable construction of reliable, domain-relevant test oracles.

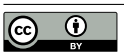
CCS Concepts

• **Software and its engineering → Software testing and debugging**.

Keywords

metamorphic testing, metamorphic relation generation, repository-level context, LLM refinement

ACM Reference Format:
Dibyendu Brinto Bose, Jiawei Qin, and Chris Brown. 2026. Repository-Aware Metamorphic Relation Generation for Augmented Reality Applications using Large Language Models. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26), October 12–16, 2026, Munich, Germany*. ACM, New York, NY, USA, 13 pages.
https://doi.org/10.1145/3832783.3837502



This work is licensed under a Creative Commons Attribution 4.0 International License.
ASE '26, Munich, Germany
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2882-2/2026/10
https://doi.org/10.1145/3832783.3837502

1 Introduction

Metamorphic Testing (MT) provides a practical solution for scenarios where defining precise test oracles is difficult. Rather than verifying single outputs, MT defines *metamorphic relations* (MRs) that describe how program outputs should change when inputs are transformed [3, 14]. Many complex software systems exhibit such relations. For example, Augmented Reality (AR) applications increasingly integrate interactions between digital content, physical environments, and code to provide users with immersive experiences across critical domains [4, 16, 28]. The output of AR programs often depends on dynamic environmental conditions, spatial information, camera input, and device motion [4, 16], making it difficult to determine expected outputs for test cases [11]. Yet, such properties can be encoded as relations to validate AR application behavior [11]. For example, when the camera moves closer to a virtual object in an AR scene, the rendered object should appear proportionally larger, and when the camera moves away it should appear smaller.

However, identifying meaningful MRs is challenging across software domains. For instance, MR generation often requires substantial domain expertise [15]. Developers must reason about how input transformations affect system behavior and how these relations can be operationalized into executable tests [34]. In large software repositories containing hundreds of classes and thousands of methods, manually identifying such relations becomes difficult and time-consuming [31].

Recent advances in large language models (LLMs) have created new opportunities to automate test specification generation [23]. Prior studies show that LLMs can assist in generating code tests and reasoning about program behavior [10, 12, 31]. Nevertheless, existing approaches typically prompt LLMs using isolated functions or short code snippets, limiting the model's ability to understand the broader architectural context in which a method operates [17, 39]. Another challenge is that LLMs lack program context for testing tasks [45], causing them to generate incorrect or irrelevant test cases that fail to verify program behavior [38].

To address these challenges, recent work suggests *repository-level code understanding* aids in effective reasoning about software systems, requiring structured contextual information beyond individual files [1, 12, 21, 35]. For example, Oskooei et al. propose representing repositories through hierarchical summaries to enable LLMs to reason about large codebases while mitigating context window limitations [44]. However, increasing context introduces a new challenge: different configurations often produce competing or overlapping MRs for the same target method, varying in specificity, domain relevance, and sometimes even contradicting each other [6, 40]. Therefore, beyond generating MRs, it is necessary to

systematically reconcile these differences to obtain reliable relations. Prior work shows *debate-based refinement* reduces contradictions and redundancies in LLMs for various software engineering tasks, such as code generation [24] and issue resolution [32].

Motivated by these insights, we explore whether structured repository context can improve the generation of MRs. Critically, existing work on debate-based LLM refinement has focused on *detecting* whether a candidate relation holds for a given code snippet [9], a classification problem where ground-truth relations are assumed to exist. The present work addresses an orthogonal problem: *generating* MRs from scratch using repository-level context, where no ground-truth relation exists, and the challenge is both discovery and quality. We organize context into three levels—repository-level metadata, class-level semantics, and method-level signatures—to expose both architectural intent and localized implementation details relevant for behavioral reasoning. We also introduce a reasoning-orchestrated deliberation mechanism that identifies contradictions and redundancies among candidates and refines them into more coherent MRs. Figure 1 provides an overview of the complete pipeline. Our study is guided by the following research questions:

- **RQ1:** How does repository-level contextual granularity affect the structural validity and coverage of LLM-generated MRs?
- **RQ2:** To what extent can structured deliberation reconcile competing MR candidates generated from different context configurations?
- **RQ3:** Can debate-refined MRs serve as executable test oracles for AR applications, and how does deliberation strategy affect their quality?

We evaluate our approach on a sample of real-world AR systems due to the complexity and difficulty of defining test oracles for AR applications [4]. Our results show that hierarchical repository context improves coverage, specificity, and diversity of generated MRs while maintaining high structural validity. Importantly, the relationship between context volume and quality is not monotonic: flat retrieval (A1) degrades source traceability relative to both the method-only baseline and hierarchical context, showing that context *organization* matters as much as context *volume*. Furthermore, the proposed deliberation mechanism effectively resolves conflicts among candidates in 90% of cases, reduces exact duplicate rates from 13.3% to 1.3%, and produces more precise and context-aware relations — with merged outputs outperforming single-best selections on every quality dimension. Finally, manual inspection shows that debate-refined MRs achieve high validity (1.80/2) and testability (1.82/2), with 61.7% rated high-quality across all dimensions. Through this study, we aim to better understand how structured repository-level context influences LLM-based test specification generation and whether reasoning orchestration mechanisms can improve the reliability of generated relations. The main contributions of this paper are:

- An empirical study demonstrating how repository-level contextual granularity affects the coverage, validity, and diversity of generated MRs for AR applications, revealing that context organization matters as much as context volume.
- A reasoning-orchestrated deliberation framework that resolves contradictions and redundancies among candidate MRs and produces refined, context-aware relations.

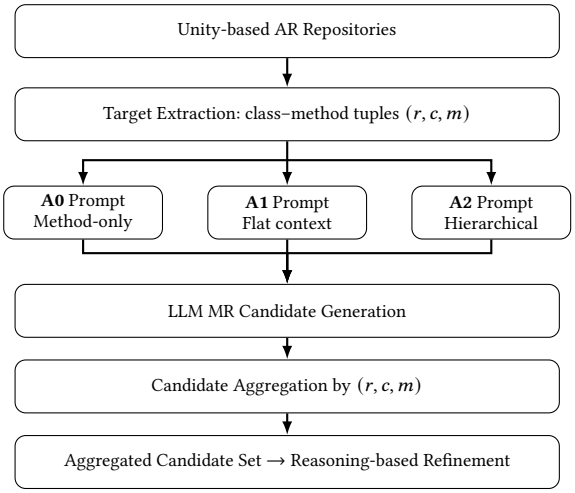


Figure 1: MR generation pipeline. For each target repository, class, and method tuple (r, c, m), prompts are constructed under A0 (method-only), A1 (flat context), and A2 (hierarchical context), passed to a shared LLM, and aggregated for downstream refinement.

- An oracle study evaluating the validity, testability, and AR-specific relevance of curated MRs, providing insights into their usefulness as test specifications.

2 Background

2.1 Challenges in Identifying MRs

Identifying MRs remains a challenging task. Constructing meaningful MRs typically requires deep understanding of both the application domain and the implementation of the software system [14]. Developers must determine which input transformations are meaningful, how outputs should change under those transformations, and how such relations can be operationalized into executable tests [5, 13]. The difficulty increases in large software repositories. Modern systems often contain hundreds of classes and thousands of methods, making it difficult for developers to systematically identify behavioral relations across different parts of the system [43, 46]. Moreover, many relations depend on architectural context, such as interactions between classes or how specific methods manipulate spatial data within the AR pipeline [15].

Prior approaches leverage search-based [59], pattern-based [49], and mutation-based [50, 52] approaches. Recent advances in LLMs have created new opportunities to assist developers in generating testing artifacts such as test cases and specifications [17, 54]. Prior work has investigated using LLMs to determine whether a given code snippet conforms to predefined MRs [9], including deducing MR input transformations for MT [57]. However, these approaches assume that MRs are already known and therefore do not address the problem of discovering new relations directly from repository-level program context.

In contrast, our work addresses the previously underexplored problem of *MR discovery from repository-level program context*. Unlike prior approaches that operate on isolated snippets or predefined relations, we leverage structured repository context to enable LLMs

to reason about behavioral properties across classes, methods, and dependencies. Furthermore, we introduce a reasoning-orchestrated deliberation mechanism to reconcile competing candidate MRs, a step that is absent in existing MR generation pipelines. This combination of *context-aware MR generation* and *deliberative refinement* allows us to produce more specific, domain-relevant, and operationalizable relations for AR systems.

2.2 Testing Challenges in AR Systems

Our work explores repository-level context for MR generation in the context of AR programs. AR applications operate in highly dynamic environments where program behavior depends not only on internal logic but also on external sensor inputs such as camera frames, spatial tracking, and device orientation [16, 28]. These characteristics introduce testing challenges that are less common in traditional software systems. For example, the output of an AR application may depend on environmental conditions such as lighting, surface detection, and camera movement, making it difficult to define precise expected outputs for test cases [4]. When tests can be implemented, prior work shows testing code for mixed reality applications suffers from substantial test smells [19].

A persistent challenge for AR systems is the *test oracle problem*, where determining the correct output for a given input is challenging or infeasible [18]. While exact outputs are often unknown, developers may still reason about expected *relationships* between inputs and outputs under controlled transformations; however, such reasoning is often informal, incomplete, and difficult to systematically capture [4, 51]. Recent work exploring model-based testing for virtual reality programs demonstrates enhanced capabilities for testing coverage, yet limitations for detecting real-world bugs [61]. This observation motivates the use of *metamorphic testing*, a technique that validates relationships between program behaviors rather than comparing a single output against a fixed oracle.

2.3 Metamorphic Testing for AR Applications

Metamorphic testing has been widely applied across domains where reliable test oracles are difficult to obtain, including scientific computing, machine learning systems, robotics, and image processing applications [27, 29, 40, 41]. In these contexts, MT verifies whether expected relations between multiple program executions hold under specific input transformations. Prior work has also explored applying MT to AR systems. Bose et al. [11] conducted a manual empirical analysis of mobile AR applications and identified seven domain-specific MRs capturing key spatial behaviors such as object scaling, occlusion consistency, and orientation stability. This study highlights that AR pipelines involve complex interactions between spatial tracking and virtual object manipulation, making behavioral properties such as MRs particularly suitable for validating AR functionality when traditional oracle-based testing becomes impractical.

3 Approach

3.1 Context Configurations for MR Generation

Recent work on repository-level code understanding shows that structured, hierarchical representations enable LLMs to reason more effectively about large codebases while mitigating context limitations [21, 35, 44]. In object-oriented systems such as Unity-based AR applications, program behavior often depends on interactions across classes and components [7, 34]. These observations motivate the use of structured repository context for MR generation.

To investigate how contextual granularity influences the quality of generated MRs, we design three configurations that expose progressively richer program information to the language model. These configurations form an ablation over context, allowing us to analyze how different levels of structural information impact the validity, specificity, and diversity of generated relations.

A0: Method-Only Context. The A0 configuration represents the minimal context condition. In this setting, the language model receives only the method signature of the target function. No information about the surrounding class or repository is provided. The goal of this configuration is to measure how well an LLM can infer potential behavioral relations using only the local function interface. While this minimal prompt reduces contextual noise, it limits the model's ability to reason about program semantics beyond the method boundary.

A1: Flat Repository Context. The A1 configuration augments the method signature with a flat set of repository artifacts retrieved from the codebase. These artifacts include the repository name and type, a list of Unity-related features used in the project, class field declarations, and a set of additional class snippets selected based on method similarity. All retrieved context is provided as a single unstructured block within the prompt. This configuration allows the model to observe broader repository signals but does not explicitly encode structural relationships between components.

A2: Hierarchical Repository Context. The A2 configuration introduces a structured representation of repository context designed to reflect the hierarchical organization of object-oriented systems, commonly used to implement AR applications [20]. The prompt contains three layers of contextual information. At the repository level, the model receives the repository name, project type, and an LLM-generated repository summary. At the class level, the prompt includes the class name, base class information, a generated class summary, and whether the class represents a Unity component. At the method level, the model receives the target method signature together with related classes identified through a dependency graph. This hierarchical structure exposes both architectural intent and localized implementation details, enabling the model to reason about program behavior at multiple abstraction levels.

3.2 Metamorphic Relation Generation Pipeline

Figure 1 presents the overall MR generation workflow. Starting from Unity-based mobile AR repositories, we extract target class–method tuples (r, c, m) where r denotes the repository, c the class, and m the method, and generate candidate MRs under three context settings: A0, A1, and A2. These settings differ only in the amount and structure of context exposed to the model.

Figure 2 summarizes the prompt design used in each configuration. A0 uses only the target method signature. A1 extends this with flat retrieved artifacts such as class fields, Unity features, and

MR Generation Prompt Templates

A0 (Method-Only).
Input: `{method_signature}`
Task: Generate candidate MRs for the target method.
Output: name, input_transformation, output_relation, target_class, target_method, source=A0

A1 (Flat Context).
Input: `{method_signature}`, `{class_fields}`, `{unity_features}`, `{retrieved_snippets}`
Task: Generate candidate MRs using the provided flat retrieved context.
Output: name, input_transformation, output_relation, target_class, target_method, source=A1

A2 (Hierarchical Context).
Input: `{repo_summary}`, `{class_summary}`, `{inheritance_info}`, `{dependency_relations}`, `{method_signature}`
Task: Generate candidate MRs using hierarchical repository context.
Output: name, input_transformation, output_relation, target_class, target_method, source=A2

Figure 2: Prompt structure used for candidate MR generation under the three context configurations.

code snippets. A2 further introduces hierarchical repository context, including repository summaries, class summaries, inheritance information, and dependency relations. This progression allows us to study how increasing contextual richness affects MR generation quality.

Given these prompts, the language model generates one or more candidate MRs for each target method. Each MR follows a structured schema containing fields such as `name`, `input_transformation`, `output_relation`, `target_class`, `target_method`, and `source`. The `source` field records whether the candidate originated from A0, A1, or A2.

Finally, candidates produced for the same (r, c, m) tuple are aggregated into its candidate set and passed to the downstream reasoning stage (Figure 1). This design separates MR generation from MR selection: the first stage proposes multiple candidate relations, while the next stage resolves conflicts and refines them through structured reasoning.

3.3 Reasoning-Orchestrated Debate for MR Refinement

Figure 3 illustrates the debate-based refinement process used to resolve competing candidate MRs generated from A0, A1, and A2. For each target (r, c, m), all candidate MRs are aggregated into a shared pool and passed to a critique-based reasoning stage. Table 1 provides a concrete example of this process.

Debate Input. For each target method, the system collects candidate MRs generated under different context configurations. Each candidate includes its input transformation, output relation, and source label (A0, A1, or A2). These candidates often differ in abstraction level, domain awareness, and coverage of program behavior.

Contradiction and Redundancy Analysis. We choose contradiction and redundancy as the primary critique dimensions because they correspond to the two fundamental outcomes of comparing multiple MR candidates for the same target: either the candidates disagree, or they overlap [14, 60]. This makes them the most appropriate dimensions for evaluating the debate stage. Structural

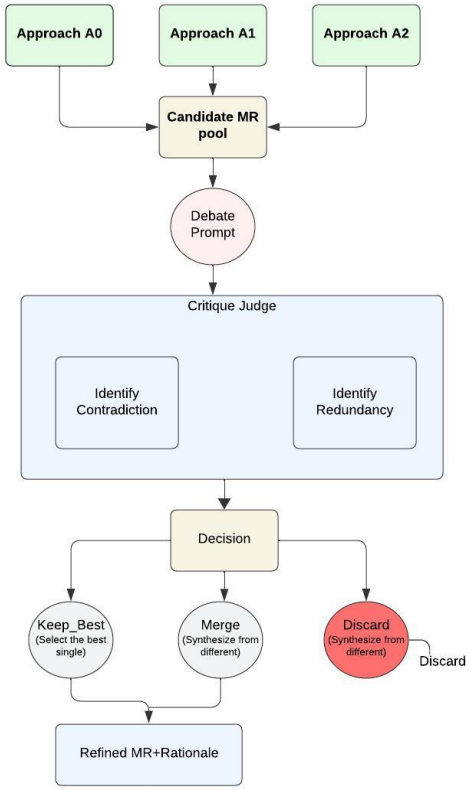


Figure 3: Debate-based MR refinement workflow. Candidate MRs generated under A0, A1, and A2 are aligned by target method, assembled into a debate prompt, evaluated by a critique/judge model, and converted into a final decision: `KEEP_BEST`, `MERGE`, or `DISCARD_ALL`.

validity, completeness, and field-level specificity are already measured at the individual-MR level in RQ1; in contrast, the debate stage must judge how candidate MRs relate to one another. For that purpose, contradiction captures unresolved conflict across candidates, while redundancy captures repeated hypotheses that add no new testing value. Together, these two dimensions directly measure whether debate is serving its intended role as a reconciliation mechanism rather than a simple selector. We validate the correctness of these decisions in Section 5.2.

- **Contradiction:** Two MRs are contradictory if they describe incompatible output relations under similar or overlapping input transformations. For example, one MR may imply invariance under a transformation while another predicts a change in behavior. Identifying such inconsistencies is essential for ensuring logical coherence in generated specifications [48].
- **Redundancy:** Two MRs are redundant if they express semantically equivalent relations with minor syntactic variation. Redundant relations do not contribute additional testing value and can reduce the diversity and usefulness of the MR set [60].

Table 1: Example of debate-based MR refinement (AR_Snake_PoseUpdate)

Target: `InstantPreviewManager.UpdateLoop()` **Repository:** `AR-Snake-Game-Using-Unity`
A0 Assessment (Method-only): The generated MRs are overly generic and lack sufficient context about the method's functionality. They do not capture the AR-specific behavior and are not precise enough to be meaningful or testable.
A1 Assessment (Flat context): The generated MR introduces a delay-based transformation, which is a plausible scenario but does not directly align with the core functionality of the `UpdateLoop` method. It also lacks specificity in how the transformation impacts the output.
A2 Assessment (Hierarchical context): The generated MR is highly specific and context-aware, correctly capturing the interaction between `InstantPreviewManager` and `InstantPreviewTrackedPoseDriver`, and how device pose updates propagate to the AR snake's movement.
Decision: `KEEP_BEST` **Best Source:** A2 **Final MR:** **Name:** AR_Snake_PoseUpdate **Input Transformation:** Apply a rotational transformation R to the device pose input provided by `InstantPreviewTrackedPoseDriver`, while keeping the virtual snake anchored to the same real-world position. **Output Relation:** (1) The snake's world-space position must remain unchanged. (2) The snake's perceived pose (position/orientation in the camera view) must change consistently with the applied rotation R. **Rationale:** The A2-generated MR is the most accurate and testable, as it reflects the core AR pipeline where real device pose updates are mapped to virtual object movement.

Critique-Based Decision. Based on detected contradictions and redundancies, the judge produces one of three decisions:

- **KEEP_BEST**: Select a single MR that best captures the target behavior.
- **MERGE**: Combine complementary aspects of multiple MRs into a refined relation.
- **DISCARD_ALL**: Remove all candidates when they are inconsistent or uninformative.

As shown in Figure 3, this process acts as a structured filtering step that eliminates conflicting or duplicate relations before producing a final MR.

Refined Output. The output of the debate stage is a refined MR accompanied by a rationale and source attribution. This refined relation is typically more context-aware and less redundant than individual candidates. Table 1 demonstrates how the debate process selects a context-aware MR (A2) over more generic alternatives, resulting in a precise and testable AR-specific relation.

Overall, the debate mechanism serves as a conflict-resolution layer that enforces consistency and reduces redundancy among candidate MRs. By explicitly reasoning over disagreements between context configurations, it produces more coherent and operationalizable relations for downstream testing.

4 Evaluation Setup

This section describes the dataset, evaluation metrics, and analysis procedures used to investigate the research questions introduced in Section 1. Our evaluation focuses on mixed methods approaches to understand how repository-level contextual information influences the structural quality and diversity of generated MRs (RQ1), how reasoning orchestration through debate refines competing MR candidates (RQ2), and the validity and usefulness of generated MRs for AR applications (RQ3).

4.1 Dataset

We use an existing dataset of open-source, executable mobile AR repositories introduced in prior work [9]. The dataset consists of Unity-based mobile AR repositories written in C# and was collected from GitHub using AR-related keywords such as *"AR Foundation"*, *"ARCore"*, *"ARKit"*, and *"Augmented Reality"*.

From repositories in this dataset, we extract class-method pairs representing potential behavioral units for analysis. In total, the dataset contains 142 repositories and 5,167 unique class–method targets. For each target method, we generate MR candidates using the three context configurations described in Section 3. Across all configurations, the generation process produced 14,916 candidate MRs, which form the basis for our experimental analysis.

4.2 LLMs Configuration

We use two open-weight instruction-tuned LLMs, as generation and deliberation impose different requirements. MR generation must scale across many repository-method instances, favoring an efficient model, whereas deliberation requires stronger reasoning to compare alternatives, resolve contradictions, and refine candidate MRs. We discuss the implications of this capability asymmetry in Section 6.

For MR generation, we use `Llama-3.2-3B-Instruct`, selected for efficient large-scale inference and strong instruction-following performance [37, 55]. We set `temperature`=0.3 and `max_new_tokens`=1024.

For refinement, we use `Qwen2.5-Coder-32B-Instruct`, leveraging its strong code reasoning ability to resolve contradictions, remove redundancies, and refine candidate MRs [22, 58]. We set `temperature`=0.3 and `max_new_tokens`=1024.

4.3 Structural Validity and Redundancy Metrics (RQ1)

To evaluate the quality of generated MRs, we adopt a set of complementary metrics capturing structural validity, specificity, diversity, and coverage. We design the structural validity metrics based on minimal criteria required for a usable MR, ensuring that each relation is well-formed, sufficiently descriptive, and actionable for defining input–output transformations [6, 14, 36]. In contrast, the remaining metrics are adapted from prior work on test generation and specification mining, where coverage, specificity, and diversity are commonly used to assess the usefulness of generated artifacts [21, 35, 44].

Data-Driven Specificity Thresholds. To avoid arbitrary heuristics when measuring textual specificity, we derive non-triviality thresholds from the pooled set of all generated MRs across A0, A1, and A2 (N = 14,916). For both the `input_transformation` and `output_relation` fields, we compute the 10th percentile (p10) character length using linear interpolation over the sorted distribution.

For a textual field f, non-triviality is defined as:

$$\text{NonTrivial}(f) = \begin{cases} 1 & \text{if } |f| \geq \text{p10} \\ 0 & \text{otherwise} \end{cases}$$

The resulting thresholds are 30 characters for `input_transformation` and 34 characters for `output_relation`. These thresholds are computed once and applied consistently across all approaches.

Structural Validity Criteria. We operationalize six binary criteria to assess whether a generated MR is structurally valid and usable for specifying testable input–output transformations. In particular, field completeness (C1), target validity (C4), and source traceability (C5) ensure that each MR can be mapped to a specific code location and generation setting. Non-triviality checks (C2–C3) enforce that both the input transformation and expected output relation contain sufficient detail to be operationalized as test assertions. Finally, the non-self-referential rationale constraint (C6) prevents degenerate cases where the model simply repeats the transformation or relation without meaningful justification.

Formally, each MR is evaluated against the following six binary criteria, where $C_i \in \{0, 1\}$:

(1) **Field Completeness (C1):** All required fields (`id`, `name`, `input_transformation`, `output_relation`, `target_class`, `target_method`, `source`) must be present and non-empty.
(2) **Input Non-Triviality (C2):** The `input_transformation` field satisfies the p10 threshold (≥ 30 characters).
(3) **Output Non-Triviality (C3):** The `output_relation` field satisfies the p10 threshold (≥ 34 characters).
(4) **Target Symbol Validity (C4):** Both `target_class` and `target_method` must be explicitly defined.
(5) **Source Traceability (C5):** The `source` field must indicate the origin configuration (A0, A1, or A2).
(6) **Non-Self-Referential Rationale (C6):** The `rationale` field must not duplicate the transformation or relation text.

Composite Structural Validity Score. To summarize structural quality, we compute a composite Structural Validity Score (SVS) as the average of the six criteria. The score ranges from 0 to 1.0, where 1.0 indicates that all structural constraints are satisfied.

$$\text{SVS}(mr) = \frac{1}{6} \sum_{i=1}^{6} C_i$$

We assign equal weight to each criterion to avoid introducing subjective bias about their relative importance. Since all criteria represent necessary (but not individually sufficient) conditions for a valid MR, equal weighting provides a neutral and interpretable aggregation that captures overall structural soundness without overfitting to any single aspect.

Redundancy Calculation. We measure redundancy using exact and near-duplicate detection based on normalized MR signatures.

Exact duplicates are defined as MRs that have identical `input_transformation` and `output_relation` after normalization. We group MRs by this pair and count duplicates by keeping one instance per group and treating the remaining instances as redundant. The exact duplicate rate is computed as the proportion of such redundant instances over the total number of MRs.

Near duplicates are defined more loosely as MRs that target the same method and use the same `input_transformation`, but may differ in their `output_relation`. We group MRs using this relaxed signature and compute the near-duplicate rate using the same counting procedure.

Coverage Metrics. We measure coverage to quantify how broadly each approach explores the program space when generating MRs. Broader coverage increases the likelihood of identifying diverse and non-overlapping behavioral relations [6, 36].

- **Total MRs Generated:** The total number of MRs reflects the overall generation capacity of an approach and its ability to propose candidate behavioral relations.
- **Repository Coverage:** The number of unique repositories indicates how well the approach generalizes across different projects and codebases.
- **Class Coverage:** The number of distinct classes captures how broadly different components within a system are explored.
- **Class–Method Coverage:** The number of unique class–method pairs provides a fine-grained view of coverage, indicating how many specific program locations are associated with generated MRs. To avoid ambiguity, methods are identified using `ClassName.methodName`.

Unique MR Rate. Finally, we compute the unique MR rate as: $\text{UniqueMRRate}(A) = 1 - \text{ExactDupRate}(A)$.

This metric reflects the proportion of structurally distinct MRs generated by each configuration.

4.4 Debate Outcome Analysis (RQ2)

To analyze how reasoning orchestration refines competing MR candidates, we examine the outcomes of the debate stage introduced in Section 3. We record the distribution of these decisions, identify which context configuration most frequently produces the winning MR, and analyze conflict signals such as contradictions or redundancies detected during debate. These measurements allow us to quantify how reasoning orchestration consolidates candidate relations into a smaller set of refined MRs.

To validate the correctness of these judge decisions, two authors independently rated a stratified random sample of 500 debate outcomes for KEEP_BEST (n = 205), MERGE (n = 213), and DISCARD_ALL (n = 82). The raters scored each decision as Correct (2), Debatable (1), or Incorrect (0) given the judge's assessments and detected conflict signals. We calculate Cohen's κ over the 500 MR decisions to measure the agreement between raters.

4.5 Test Oracle Study (RQ3)

To evaluate whether debate-refined relations can serve as practical test oracles, we conduct a manual evaluation study on a curated subset of MRs. Starting from the full set of debate-refined MRs (N = 3,760), the first author manually reviewed all candidates and selected 141 MRs that capture AR-specific behaviors. The selection is based on the semantic content of each MR, including the input

Table 2: Evaluation Criteria for Test Oracle Study

Dimension	Key Question	Evaluation Criteria
Validity	Is the MR logically correct and meaningful?	(1) Transformation defines a clear input change; (2) Output relation correctly relates source and follow-up outputs; (3) Transformation and relation are logically consistent; (4) Violations indicate real faults
Testability	Can the MR be directly implemented as a test?	(1) Transformation is programmatically applicable; (2) Output relation is expressible as an assertion; (3) Terms are unambiguous; (4) No interpretation is required for implementation
AR Specificity	Does the MR reflect AR-specific behavior?	(1) Uses AR concepts (e.g., pose, alignment, occlusion); (2) Not trivially applicable to non-AR systems; (3) Targets AR-related methods or behaviors; (4) Reflects real-world interaction or spatial logic

transformation, output relation, and associated program context (target class and method).

We note that MR names are not used as a filtering criterion, as prior work has shown that surface-level identifiers are often unreliable indicators of semantic intent in generated artifacts [13, 54]. Instead, selection follows explicit criteria grounded in established AR/3D interaction concepts [30]: MRs capturing pose propagation, spatial transformations, rendering consistency, or camera- and sensor-driven behavior are included; those involving generic state changes, UI-only logic, or behavior with no AR grounding are excluded. Full criteria and sample exclusions are in our artifact.

Evaluation Dimensions. We evaluate each MR along three dimensions: *validity*, *testability*, and *AR specificity*. These dimensions are grounded in prior work on test oracle design and specification quality, where useful oracles must be logically correct, executable, and contextually relevant [8, 33]. Note that testability here denotes a property of the MR specification itself, whether its transformation and relation can be operationalized as a programmatic assertion, rather than the testability of the system under test.

Scoring Scheme. Following standard practices in empirical software engineering for qualitative artifact evaluation, we adopt a 3-point ordinal scale (0–2) for each dimension [53]. Such coarse-grained scales are commonly used to balance annotation consistency and discriminative power in human evaluation studies [25, 47]. The scale definitions are operationalized using structured checklists, as summarized in Table 2.

Annotation Procedure. Three independent raters (Rater 1, Rater 2, and Rater 3) evaluated all 141 MRs using the predefined criteria. Each rater independently assigned scores across all dimensions without coordination. This process follows standard multi-rater evaluation protocols used in empirical software engineering studies.

Inter-Rater Agreement. To assess annotation reliability, we compute Fleiss' κ for multi-rater agreement and Krippendorff's α (ordinal) to account for disagreement magnitude. In addition, we report exact agreement and adjacent agreement (difference ≤ 1), which is appropriate for ordinal scales with skewed distributions. Pairwise weighted κ is also computed to analyze agreement between individual raters.

5 Result

5.1 RQ1: Effect of Context Granularity

Table 3: Effect of Context Granularity on Specificity and Diversity

Metric	A0	A1	A2
Input Transformation Specificity			
Mean Length	60.8	72.6	75.0
Median Length	47	63	69
Output Relation Specificity			
Mean Length	68.0	77.5	73.5
Median Length	57	72	70
Diversity			
Exact Duplicate Rate	18.9%	14.8%	13.3%
Near Duplicate Rate	10.8%	7.2%	6.7%
Unique MR Rate	81.1%	85.2%	86.7%

Table 3 and Table 4 show that hierarchical context (A2) achieves the strongest coverage, specificity, and diversity overall, Though the relationship between context volume and quality is not uniformly monotonic across all criteria.

Coverage. A2 achieves the highest coverage across all dimensions, including unique classes (474), and class–method pairs (5,167), compared to A0 and A1 (Table 4). This indicates that hierarchical context enables the model to generalize across a broader set of program elements, beyond simply generating more MRs.

Structural Validity. All configurations maintain high structural validity (SVS $\approx$ 0.96–0.97), but the relationship between context and quality is not monotonic. A1, despite providing more raw context than A0, degrades source traceability to 96.25%—the only configuration falling below 100% on this criterion. This suggests unstructured flat retrieval introduces noise that disrupts schema adherence without adding behavioral signal. Hierarchical context (A2) recovers this and goes further, achieving the highest non-triviality rates for both input (92.43%) and output relations (92.00%), indicating that structured context enables more meaningful transformations rather than merely longer outputs.

Table 4: Effect of Context Granularity on Structural Validity and Coverage

Metric	A0 (Method-Only)	A1 (Method+Class)	A2 (Hierarchical)
Coverage			
Total MRs Generated	4,046	3,866	7,004
Unique Classes Covered	326	305	474
Unique Class.Method Pairs	3,144	2,933	5,167
Overall Structural Validity			
Mean SVS	0.9624	0.9615	0.9702
% SVS = 1.0	82.45%	86.26%	85.21%
Selected Per-Criterion Pass Rates			
Input Non-Triviality(% ≥ p10 Threshold)	86.7%	90.4%	92.43%
Output Non-Triviality(% ≥ p10 Threshold)	90.78%	91.72%	92.00%
Target Validity	100%	100%	100%
Source Traceability	100%	96.25%	100%

Specificity and Diversity. As shown in Table 3, A2 improves input specificity (highest mean and median lengths, and 92.4% above the threshold) while maintaining competitive output specificity. In addition, A2 reduces redundancy (lowest exact and near-duplicate rates) and achieves the highest unique MR rate (86.7%), indicating more diverse and distinct MRs.

RQ1 Summary

Hierarchical context (A2) achieves the highest coverage (5,167 class–method pairs) and diversity (86.7% unique MRs) while maintaining high structural validity (SVS ≈ 0.96). We also observed flat retrieval (A1) degrades source traceability to 96.25%—below both A0 and A2—showing that context *organization* matters as much as context *volume*.

5.2 RQ2: Effect of Multi-Agent Debate

Table 5: RQ2: Conflict Identification and Resolution in Multi-Agent Debate (N = 4496)

Metric	Count	Rate
Contradictions detected	3550	79.0%
Redundancies detected	2301	51.2%
No issues detected	442	9.8%

Table 5, Table 6, and Table 7 summarize the role of multi-agent debate in identifying conflicts and improving MR quality.

Conflict Identification and Resolution. Deliberation was applied only to the 4,496 targets whose candidate MRs originated from at least two context configurations, since debate requires competing candidates; single-source targets were excluded. As shown in Table 5, conflicts are prevalent among candidate MRs, with contradictions detected in 79.0% and redundancies in 51.2% of cases, while only 9.8% exhibit no issues. This highlights that independently generated MRs frequently disagree or overlap.

Table 6: RQ2: Winning Source Distribution After Debate (N = 3760 (4496 − 736 DISCARD_ALL))

Source	Count	Rate
A2 (Hierarchical)	3316	88.2%
MERGED	388	10.3%
A1	47	1.2%
A0	9	0.2%

Preference for Context-Rich MRs. Table 6 shows that the debate process overwhelmingly favors MRs generated with hierarchical context. A2 accounts for 88.2% of final selections, while A1 and A0 contribute minimally (1.2% and 0.2%, respectively). Additionally, 10.3% of cases result in merged outputs, suggesting that combining complementary information across sources can further improve MR quality. These results demonstrate that richer contextual information consistently leads to more reliable and preferred MRs.

Impact on MR Quality. The end-to-end impact of debate is shown in Table 7. Debate-refined MRs are more detailed, with higher average and median lengths for both input transformations (89.3 / 83.0) and output relations (101.2 / 95.0) compared to all individual configurations. At the same time, redundancy is substantially reduced, with the exact duplicate rate dropping from 13.3% (A2) to 1.3%. Although the total number of MRs decreases to 3,760 (the 4,496 debate groups minus 736 DISCARD_ALL decisions), this reflects effective filtering and consolidation rather than loss of useful information.

Manual Validation of Judge Decisions. The two raters agreed on 94.2% of decisions (κ = 0.865; weighted κ = 0.87), with all disagreements adjacent on the scale (Correct vs. Debatable). Only 0.6% (n = 3) of decisions were flagged as Incorrect by either rater; the remainder split between clearly correct (69%) and debatable (31%). KEEP_BEST (94–98%) and DISCARD_ALL (89–94%) were overwhelmingly rated clearly correct, indicating the judge is trustworthy for comparative and dismissive decisions. MERGE is the outlier: only 31–37% of MERGE decisions were rated unambiguously correct. The majority of these were due to the LLM judge making the final decision to MERGE when KEEP_BEST would have been the simpler outcome.

RQ2 Summary

Multi-agent debate resolves conflicts in 90% of cases and overwhelmingly selects A2-derived MRs (88.2%), reducing duplicate rates from 13.3% to 1.3% while producing more detailed and refined relations.

5.3 RQ3: Generated MR Practicality

To examine the suitability of generated MRs for testing real-world AR programs, we conducted a manual oracle study.

Inter-Rater Reliability. Three raters independently evaluated 141 debate-refined MRs on validity, AR specificity, and testability. Overall agreement is in the slight-to-fair range, with Fleiss' κ values of 0.33 for validity, 0.22 for AR specificity, and 0.19 for testability. However, these values should be interpreted together with the high adjacent agreement rates of 94.3%, 95.7%, and 96.5%, respectively, indicating that disagreements were usually limited to a one-level difference on the 0–2 scale. Pairwise weighted agreement reveals a clear structure: Rater 1 and Rater 2 are strongly aligned across all three dimensions (κ_w = 0.73, 0.73, and 0.58), whereas agreement involving Rater 3 is much lower, especially for testability ($\kappa_w = -0.01$ between Rater 1 and Rater 3). This pattern suggests that the lower multi-rater coefficients are driven less by random disagreement and more by a systematic difference in rating strictness, particularly for AR specificity and testability.

Quantitative Findings. Table 8 shows that validity and testability are the strongest dimensions, with mean scores of 1.80 and 1.82, and majority-high rates of 86.5% and 88.0%, respectively. This indicates that most debate-refined MRs are both logically sound and sufficiently well-specified for human raters to judge them as directly implementable as test conditions without requiring additional interpretation. AR specificity is lower and more variable (mean = 1.68, SD = 0.45), which is consistent with the fact that the curated set spans both AR-core behaviors and application-layer logic. Even so, 82.9% of MRs receive majority-high AR-specificity scores. Across all three dimensions, 87 of 141 MRs (61.7%) satisfy the high-quality criterion, and 35 (24.8%) achieve top-tier quality across all dimensions.

Effect of Deliberation Strategy. The debate strategy also affects oracle quality. MRs produced through MERGE outperform KEEP-BEST on every dimension, with a higher composite score (1.81 vs. 1.73), higher validity (1.85 vs. 1.77), higher AR specificity (1.70 vs. 1.66), and higher testability (1.87 vs. 1.78). This indicates that synthesizing information from competing candidates yields stronger oracle specifications than simply retaining a single candidate.

RQ3 Summary

Debate-refined MRs achieve high validity (1.80) and testability (1.82), with 61.7% rated high-quality across all dimensions, indicating that most generated relations are sufficiently valid and well-specified to serve as practical test oracle specifications.

6 Discussion

Based on our findings, we provide implications for LLM MR generation, testing AR applications, and metamorphic testing in practice.

6.1 Implications for LLM-Based MR Generation

Our results surface implications for LLM-based MR generation that go beyond the intuition that "more context is better".

Context Configurations. We observed flat retrieval (A1) actually *degrades* source traceability below both the method-only baseline (A0) and the hierarchical configuration (A2), dropping from 100% to 96.25% (Table 4), failing to map generated MRs to the source code. This occurs because unstructured retrieval injects loosely related snippets that disrupt schema adherence without adding behavioral signal. Hierarchical context (A2) recovers this and additionally improves specificity (92.4% non-trivial input transformations vs. 86.7% for A0) and diversity (86.7% unique MR rate vs. 81.1%). Taken together, these results suggest that *how* context is organized matters as much as *how much* context is provided.

Repository-level LLM tasks more broadly —i.e., code search, bug localization, specification mining — face the same tension between providing sufficient context and maintaining structural coherence [35, 44]. Our ablation shows hierarchical organization, which preserves relationships between repository, class, and method layers, outperforms flat retrieval, even when the flat approach has access to the same underlying artifacts. LLM-based MR generation pipelines over large codebases should treat context structure as a first-class design decision, not an implementation detail.

Debate-Based Refinement. A second key finding is that MR generation is inherently a multi-perspective problem. Candidates produced independently under A0, A1, and A2 exhibit contradictions in 79.0% of cases and redundancies in 51.2%, confirming that no single context view captures the full behavioral surface of a target method. Single-pass generation pipelines, regardless of context richness, will therefore produce unreliable candidate sets. Reasoning-based deliberation is not an optional refinement step; it is structurally necessary. The debate mechanism reduces exact duplicates from 13.3% to 1.3% and consistently selects or synthesizes context-aware relations, with merged outputs outperforming single-best selections on every quality dimension in our oracle study (composite score 1.81 vs. 1.73).

6.2 Implications for Practitioners

Despite strong overall results, 38.3% of evaluated MRs did not meet our high-quality threshold across all three dimensions, reflecting two practical realities practitioners should anticipate.

Human-in-the-loop. MRs passing structural validity checks can still be too abstract for direct instantiation, particularly for application-layer behaviors lacking clear AR-specific semantics. AR specificity is the most variable dimension (mean 1.68, SD 0.45), confirming the pipeline is most reliable for core AR behaviors like pose propagation, spatial transformations, rendering consistency and less so for higher-level logic. Practitioners should treat output as a ranked candidate set requiring lightweight human review.

Model asymmetry and generalizability. We intentionally pair a smaller generation model (Llama-3.2-3B) with a larger deliberation model (Qwen2.5-Coder-32B) to balance scalability with reasoning

Table 7: End-to-End Impact of Debate on MR Quality

Metric	A0	A1	A2	Debate-Refined
Total MRs	4046	3866	7004	3760
Avg Transformation Length	60.8	72.6	75.0	**89.3**
Median Transformation Length	47.0	63.0	69.0	**83.0**
Avg Relation Length	68.0	77.5	73.5	**101.2**
Median Relation Length	57.0	72.0	70.0	**95.0**
Exact Duplicate Rate	18.9%	14.8%	13.3%	**1.3%**

Table 8: Quality of Debate-Refined MRs

Dimension	Mean	SD	Majority High	Consensus High
Validity	1.80	0.35	86.5%	67.4%
AR Specificity	1.68	0.45	82.9%	48.2%
Testability	1.82	0.29	88.0%	63.8%
Composite	1.77	0.23	–	–

quality. This raises the question of whether the judge's strong preference for A2 (88.2%) partly reflects shared biases with hierarchical prompting. The MERGE outcomes (10.3%), whose merged MRs consistently outperform KEEP_BEST despite diluting the A2 signal, and the independent human oracle study both provide counterevidence. Future work should evaluate deliberation with matched-capability models to isolate context structure from model capacity. Although our pipeline is domain-agnostic in design, all 142 repositories are Unity-based C# projects; extension to domains such as robotics or embedded systems remains a necessary validation step.

6.3 Execution-Based Validation

To examine whether generated MRs can be operationalized as executable tests and whether they detect injected faults, we conducted a preliminary case study focused on execution-based validation.

Method: We randomly selected five debate-refined MRs from different AR repositories. The first author translated each natural language relation into property-based assertions over independently checkable properties, reviewed by a second author. For each MR, we executed 10,000 randomly generated test cases on the correct implementation and on mutated versions of the target method. Following prior work that uses mutation analysis to evaluate generated MRs across domains [26, 42, 56], we applied four commonly used mutation operators: Arithmetic Operator Replacement (AOR), Unary Operator Insertion (UOI), Scalar Variable Replacement (SVR), and Statement Deletion (SDL). These mutations were applied mechanically, without using knowledge of the MR properties. Across the five randomly selected MRs, we applied 30 mutants in total.

Figure 4 illustrates one such translation for `Frame._Draw2DBody()`. The generated MR perturbs GL-space coordinates and expects the resulting world-space position to change consistently under a fixed camera configuration and depth. Rather than treating the MR only as a semantic relation, we operationalize it as a reusable property-based test over transformed inputs and relational output assertions.

Illustrative MR-to-PBT Translation

Target Method: `Frame._Draw2DBody()`

Relevant Code Fragment:

```
Vector3 glCoord = pair.Value.Coordinate2D;
Vector3 worldCoord = new Vector3((glCoord.x + 1) / 2,
    (glCoord.y + 1) / 2, 3);
m_skeletonPointObject[(int)pair.Key].transform.position =
    m_skeletonCamera.ViewportToWorldPoint(worldCoord);
```

Generated MR:

Input Transformation: Perturb the input GL coordinates (x, y) by a bounded non-zero offset $(\delta x, \delta y)$ such that both the original and perturbed coordinates remain within the valid range $[-1, 1]^2$.

Output Relation: For a fixed camera configuration and fixed depth $z = 3$, perturbing the GL-space input should produce a corresponding displacement in the world-space position returned by `ViewportToWorldPoint`. The displacement should be directionally consistent with the perturbation, and repeated equal perturbation steps should yield approximately equal changes in world-space position.

Derived PBT Property:

```
For all (x,y) in [-1,1]^2 and (dx,dy) such that
(x+dx, y+dy) and (x+2dx, y+2dy) remain in [-1,1]^2:

  p   = ViewportToWorld((x+1)/2,      (y+1)/2,      3)
  p'  = ViewportToWorld((x+dx+1)/2,   (y+dy+1)/2,   3)
  p'' = ViewportToWorld((x+2dx+1)/2,  (y+2dy+1)/2,  3)

  assert ||p' - p|| > 0 when (dx,dy) != (0,0)
  assert ||(p'' - p') - (p' - p)|| < epsilon
```

Figure 4: Example of translating a generated MR into a property-based testing specification.

Results: Table 9 summarizes how each MR was converted into executable properties and what types of injected faults those properties detected. The generated property-based tests killed 16 out of 30 mutants (53%). The remaining 14 mutants did not violate the corresponding MR properties because they preserved the relational behavior being tested. For example, some mutations changed absolute offsets, centering constants, waveform details, or isometric structure, but did not change the input-output relation specified by the MR. These cases were therefore treated as equivalent with respect to the tested metamorphic property. **After excluding such equivalent mutants, all non-equivalent mutants (16/16, 100%) were detected**.

Implication: These results provide preliminary evidence that the generated MRs are not only structurally valid and human-rated as testable, but can also be translated into executable tests that detect seeded behavioral faults such as wrong-sign arithmetic, axis confusion, variable substitution errors, scaling errors, etc.

Table 9: Translation of five generated MRs into executable properties and mutation checks.

MR Type	Executable Properties	Detected Mutations / Fault Types
GL-coordinate perturbation	Non-zero GL perturbation should produce non-zero world-space displacement; equal GL perturbation steps should produce equal world-space steps; world-space displacement direction should match GL-space perturbation; single-axis perturbation should not affect the orthogonal axis.	Killed mutations that negated `gl_x`, negated `gl_y`, or replaced `gl_x` with `gl_y`; detects wrong-sign arithmetic and axis-confusion faults.
Position reset	Moving an object by $+v$ and then $-v$ should return it to the original position; zero displacement should preserve position; sequential moves should compose additively; displacement magnitude should be preserved.	Killed mutations that scaled one displacement component, substituted one axis for another, or removed the update on one axis; detects scaling, variable substitution, and missing-update faults.
Rotation magnitude scaling	Scaling the shake amount by factor k should scale the resulting rotation by k; zero amount should produce zero rotation; larger input magnitude should produce larger rotation magnitude; axis-specific input should affect only the corresponding rotation axis.	Killed arithmetic mutations replacing multiplicative scaling with additive updates; detects violations of proportionality in rotation response.
Camera position transformation	Moving the camera by displacement d should shift each trackable's camera-relative position by $-d$; zero camera movement should preserve relative position; sequential translations should compose additively; trackable world-space position and inter-trackable distances should remain invariant.	Killed mutations changing subtraction to addition, negating the relative position, dropping a camera-coordinate update, returning camera position instead of trackable position, or scaling the relative vector; detects incorrect coordinate-frame transformations.
Raycast position shift	A small screen-coordinate shift should produce a bounded movement in the 3D hit point; zero shift should be deterministic; hit displacement should remain ratio-consistent across shift magnitudes; X-direction screen shifts should move the hit point consistently in world space; doubling the shift should approximately double the hit displacement.	Killed mutations that negated the ray's X direction or substituted `screen_y` for `screen_x`; detects wrong-direction ray mapping and screen-axis confusion.

7 Threats to Validity

Internal Validity. Our structural metrics rely on design choices such as p10-based thresholds and equal weighting in SVS. We mitigate this by deriving thresholds from the pooled dataset and reporting both aggregate and per-criterion results. The deliberation stage additionally depends on LLM reasoning, which may introduce variability; we address this by reporting decision distributions and conflict signals across a large number of debates.

Construct Validity. The three oracle dimensions (validity, testability, AR specificity) may not capture all aspects of usefulness in real-world development workflows. We mitigate this by grounding them in prior work [8, 33] and using structured scoring checklists (Table 2) to ensure consistent interpretation.

External Validity. Results are based on 142 Unity C# repositories and two LLMs; generalizability to other domains beyond AR and models is limited. We mitigate this by designing the pipeline to be domain- and LLM-agnostic. Our preliminary case study is also limited to five repositories, which future work will scale.

Conclusion Validity. The oracle study involves three raters and an ordinal scale, which may introduce subjectivity. We mitigate this with multiple agreement metrics (Fleiss' κ, Krippendorff's α, exact and adjacent agreement) and by analyzing score distributions to account for skewed ratings.

8 Future Work

This work opens several future research directions. First, generated MRs can be automatically translated into executable test cases (e.g., property-based or unit tests), enabling end-to-end integration into testing pipelines. Incorporating runtime feedback (e.g., test failures or execution traces) could further refine and validate generated relations. Extending to other domains that often leverage metamorphic testing beyond AR (e.g., robotics [29] or simulation systems [2]) and incorporating other models in our evaluation pipeline (e.g., GPT-4 or Claude) would help assess the generalizability of our approach. Finally, improving domain grounding—such as leveraging API semantics or engine-specific knowledge—may further enhance the precision of generated MRs for application-layer behaviors.

9 Conclusion

We present a repository-aware pipeline for generating and refining MRs through structured context and reasoning-based deliberation. We evaluate our approach on 142 real-world AR codebases. Our results show hierarchical repository context improves coverage and diversity, while deliberation effectively resolves conflicts and reduces redundancy. Human inspection further demonstrates refined MRs are largely valid, testable, and usable as test oracles, and a preliminary case study ($n = 5$) shows generated MRs can be translated into property-based tests that surface bugs in real-world AR code. These findings highlight the potential of combining repository-level context with reasoning orchestration to enable scalable and reliable oracle generation for complex software systems.

10 Data Availability

All data and study artifacts, including our evaluation pipeline scripts, prompt templates, etc., are publicly available to support reproducibility and reuse.[1]

References

[1] Sabiha Afroz, Redwan Ibne Seraj Khan, Hadeel Albahar, Jingoo Han, and Ali R Butt. 2025. 10Cache: Heterogeneous Resource-Aware Tensor Caching and Migration for LLM Training. In *Proceedings of the 2025 ACM Symposium on Cloud Computing*. 320–333. doi:10.1145/3772052.3772236

[2] John Ahlgren, Maria Berezin, Kinga Bojarczuk, Elena Dulskyte, Inna Dvortsova, Johann George, Natalija Gucevska, Mark Harman, Maria Lomeli, Erik Meijer, et al. 2021. Testing web enabled simulation at scale using metamorphic testing. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 140–149. doi:10.1109/ICSE-SEIP52600.2021.00023

[3] Emran Altamimi, Abdullah Elkawakjy, and Cagatay Catal. 2023. Metamorphic relation automation: Rationale, challenges, and solution directions. *Journal of Software: Evolution and Process* 35, 1 (2023), e2509. doi:10.1002/smr.2509

[4] Narges Ashtari, Andrea Bunt, Joanna McGrenere, Michael Nebeling, and Parmit K. Chilana. 2020. Creating Augmented and Virtual Reality Applications: Current Practices, Challenges, and Opportunities. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) *(CHI '20)*. Association for Computing Machinery, New York, NY, USA, 1–13. doi:10.1145/3313831.3376722

[5] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021). doi:10.48550/arXiv.2108.07732

[6] Jinsheng Ba, Yuancheng Jiang, and Manuel Rigger. 2025. Metamorphic Coverage. *arXiv preprint arXiv:2508.16307* (2025). doi:10.48550/arXiv.2508.16307

[7] Sebastian Baltes, Florian Angermeir, Chetan Arora, Marvin Muñoz Barón, Chunyang Chen, Lukas Böhme, Fabio Calefato, Neil Ernst, Davide Falessi, Brian Fitzgerald, et al. 2025. Guidelines for empirical studies in software engineering involving large language models. *arXiv preprint arXiv:2508.15503* (2025). doi:10.48550/arXiv.2508.15503

[8] Earl T Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. 2014. The oracle problem in software testing: A survey. *IEEE transactions on software engineering* 41, 5 (2014), 507–525. doi:10.1109/TSE.2014.2372785

[9] Dibyendu Brinto Bose, Yoseph Berhanu Alebachew, and Chris Brown. 2025. LLMs in Debate: Does Arguing Make Them Better at Detecting Metamorphic Relations?. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*. 43–50. doi:10.1109/ASEW67777.2025.00019

[10] Dibyendu Brinto Bose, Travis Chan, Matthew Trimble, and Chris Brown. 2025. AutoPyDep: A Recommendation System for Python Dependency Management Utilizing Graph-Based Analytics. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering* (Clarion Hotel Trondheim, Trondheim, Norway) *(FSE Companion '25)*. Association for Computing Machinery, New York, NY, USA, 1035–1039. doi:10.1145/3696630.3728576

[11] Dibyendu Brinto Bose, Brendan David-John, and Chris Brown. 2024. Optimizing ar application testing: Integrating metamorphic testing to address developer and end-user challenges. In *International Conference on Human-Computer Interaction*. Springer, 20–33. doi:10.1007/978-3-031-76812-5_2

[12] Arda Celik and Qusay H Mahmoud. 2025. A Review of Large Language Models for Automated Test Case Generation. *Machine Learning and Knowledge Extraction* 7, 3 (2025), 97. doi:10.3390/make7030097

[13] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021). doi:10.48550/arXiv.2107.03374

[14] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, TH Tse, and Zhi Quan Zhou. 2018. Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys (CSUR)* 51, 1 (2018), 1–27. doi:10.1145/3143561

[15] Stevão Alves de Andrade, Fatima LS Nunes, and Márcio Eduardo Delamaro. 2023. Exploiting deep reinforcement learning and metamorphic testing to automatically test virtual reality applications. *Software Testing, Verification and Reliability* 33, 8 (2023), e1863. doi:10.1002/stvr.1863

[16] M Eswaran and M V A Raju Bahubalendruni. 2022. Challenges and opportunities on AR/VR technologies for manufacturing systems in the context of industry 4.0: A state of the art review. *Journal of Manufacturing Systems* 65 (2022), 260–278. doi:10.1016/j.jmsy.2022.09.016

[1] https://github.com/brintodibyendu/MR_DISCOVERY_ASE

[17] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M Zhang. 2023. Large language models for software engineering: Survey and open problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. IEEE, 31–53.

[18] Maribeth Gandy and Blair MacIntyre. 2014. Designer's Augmented Reality Toolkit, Ten Years Later: Implications for New Media Authoring Tools. In *Proceedings of the 27th Annual ACM Symposium on User Interface Software and Technology* (Honolulu, Hawaii, USA) *(UIST '14)*. Association for Computing Machinery, New York, NY, USA, 627–636. doi:10.1145/2642918.2647369

[19] Faraz Gurramkonda, Avishak Chakroborty, Bruce Maxim, Mohamed Wiem Mkaouer, and Foyzul Hassan. 2025. VRTestSniffer: Test Smell Detector for Virtual Reality (VR) Software Projects. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1956–1968. doi:10.1109/ASE63991.2025.00163

[20] Lai Yip Hang, Rohaida Romli, and Nadia Diyana Mohd Muhaiyuddin. 2023. AR-OOP: Augmented Reality-based Mobile Learning Application for Learning Object-Oriented Programming. *Multidisciplinary Applied Research and Innovation* 4, 1 (2023), 116–121. doi:10.30880/mari.2023.04.01.022

[21] Anh Nguyen Hoang, Minh Le-Anh, Bach Le, and Nghi D. Q. Bui. 2026. CodeWiki: Evaluating AI's Ability to Generate Holistic Documentation for Large-Scale Codebases. (July 2026), 5812–5827. doi:10.18653/v1/2026.findings-acl.288

[22] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024). doi:10.48550/arXiv.2409.12186

[23] Sangwon Hyun, Mingyu Guo, and M Ali Babar. 2024. Metal: Metamorphic testing framework for analyzing large-language model qualities. In *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 117–128. doi:10.1109/ICST60714.2024.00019

[24] Md. Ashraful Islam, Mohammed Eunus Ali, and Md Rizwan Parvez. 2025. CodeSim: Multi-Agent Code Generation and Problem Solving through Simulation-Driven Planning and Debugging. In *Findings of the Association for Computational Linguistics: NAACL 2025*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 5128–5154. doi:10.18653/v1/2025.findings-naacl.285

[25] Lingxiao Jiang, Ghassan Misherghi, Zhendong Su, and Stephane Glondu. 2007. Deckard: Scalable and accurate tree-based detection of code clones. In *29th International Conference on Software Engineering (ICSE'07)*. IEEE, 96–105. doi:10.1109/ICSE.2007.30

[26] René Just and Franz Schweiggert. 2009. Evaluating testing strategies for imaging software by means of mutation analysis. (2009), 205–209. https://pdfs.semanticscholar.org/5381/223e2ae46a77f85cb5f348723b61f0020200.pdf

[27] Upulee Kanewala and James M. Bieman. 2013. Using machine learning techniques to detect metamorphic relations for programs without test oracles. In *2013 IEEE 24th International Symposium on Software Reliability Engineering (ISSRE)*. 1–10. doi:10.1109/ISSRE.2013.6698899

[28] Veronika Krauß, Alexander Boden, Leif Oppermann, and René Reiners. 2021. Current Practices, Challenges, and Design Implications for Collaborative AR/VR Application Development. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems* (Yokohama, Japan) *(CHI '21)*. Association for Computing Machinery, New York, NY, USA, Article 454, 15 pages. doi:10.1145/3411764.3445335

[29] Thomas Laurent, Paolo Arcaini, Xiao-Yi Zhang, and Fuyuki Ishikawa. 2024. Metamorphic testing of an autonomous delivery robots scheduler. In *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 361–372. doi:10.1109/ICST60714.2024.00040

[30] Joseph J LaViola Jr, Ernst Kruijff, Ryan P McMahan, Doug Bowman, and Ivan P Poupyrev. 2017. 3D user interfaces: theory and practice. Addison-Wesley Professional.

[31] Haiyang Li. 2025. Mrg-bench: Evaluating and exploring the requirements of context for repository-level code generation. *arXiv preprint arXiv:2508.02998* (2025). doi:10.48550/arXiv.2508.02998

[32] Han Li, Yuling Shi, Shaoxin Lin, Xiaodong Gu, Heng Lian, Xin Wang, Yantao Jia, Tao Huang, and Qianxiang Wang. 2025. Swe-debate: Competitive multi-agent debate for software issue resolution. *arXiv preprint arXiv:2507.23348* (2025). doi:10.48550/arXiv.2507.23348

[33] Nan Li and Jeff Offutt. 2016. Test oracle strategies for model-based testing. *IEEE Transactions on Software Engineering* 43, 4 (2016), 372–395. doi:10.1109/TSE.2016.2597136

[34] Rui Li, Huai Liu, Pak-Lok Poon, Dave Towey, Chang-Ai Sun, Zheng Zheng, Zhi Quan Zhou, and Tsong Yueh Chen. 2025. Metamorphic relation generation: State of the art and research directions. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–25. doi:10.1145/3708521

[35] Xiangyan Liu, Bo Lan, Zhiyuan Hu, Yang Liu, Zhicheng Zhang, Fei Wang, Michael Qizhe Shieh, and Wenmeng Zhou. 2025. Codexgraph: Bridging large language models and code repositories via code graph databases. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 142–160. doi:10.18653/v1/2025.naacl-long.7

[36] Yanwen Liu, Ruifeng Li, Hao Tao, and Zheng Zheng. 2024. Test Adequacy Criteria for Metamorphic Testing. In *2024 IEEE 24th International Conference on*

Software Quality, Reliability, and Security Companion (QRS-C). IEEE, 527–534. doi:10.1109/QRS-C63300.2024.00072

[37] Junyi Lu, Lei Yu, Xiaojia Li, Li Yang, and Chun Zuo. 2023. Llama-reviewer: Advancing code review automation with large language models through parameter-efficient fine-tuning. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 647–658. doi:10.1109/ISSRE59848.2023.00026

[38] Noble Saji Mathews and Meiyappan Nagappan. 2024. Design choices made by LLM-based test generators prevent them from finding bugs. *arXiv preprint arXiv:2412.14137* (2024). doi:10.48550/arXiv.2412.14137

[39] Noble Saji Mathews and Meiyappan Nagappan. 2024. Test-driven development and llm-based code generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1583–1594. doi:10.1145/3691620.3695527

[40] Johannes Mayer and Ralph Guderlei. 2006. An Empirical Study on the Selection of Good Metamorphic Relations. In *30th Annual International Computer Software and Applications Conference (COMPSAC'06)*, Vol. 1. 475–484. doi:10.1109/COMPSAC.2006.24

[41] Christian Murphy, Gail Kaiser, Lifeng Hu, and Leon Wu. 2008. Properties of Machine Learning Applications for Use in Metamorphic Testing. 867–872. doi:10.7916/D8XK8PFD

[42] Christian Murphy, Mohammad S Raunak, Andrew King, Sanjian Chen, Christopher Imbriano, Gail Kaiser, Insup Lee, Oleg Sokolsky, Lori Clarke, and Leon Osterweil. 2011. On effective testing of health care simulation software. In *Proceedings of the 3rd workshop on software engineering in health care*. 40–47. doi:10.1145/1987993.1988003

[43] Islam Nasr, Lobna Nassar, and Fakhri Karray. 2024. A study of the interactive role of metamorphic testing and machine learning in the quality assurance of a deep learning forecasting application. *International Journal of Information Technology* 16, 1 (2024), 105–120. doi:10.1007/s41870-023-01390-9

[44] Amirkia Rafiei Oskooei, Selcan Yukcu, Mehmet Cevheri Bozoglan, and Mehmet S Aktas. 2025. Repository-level code understanding by llms via hierarchical summarization: Improving code search and bug localization. In *International Conference on Computational Science and Its Applications*. Springer, 88–105. doi:10.1007/978-3-031-97576-9_6

[45] Nikitha Rao, Kush Jain, Uri Alon, Claire Le Goues, and Vincent J. Hellendoorn. 2023. CAT-LM Training Language Models on Aligned Code And Tests. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 409–420. doi:10.1109/ASE56229.2023.00193 ISSN: 2643-1572.

[46] Faqeer Ur Rehman and Madhusudan Srinivasan. 2023. Metamorphic testing for machine learning: Applicability, challenges, and research opportunities. In *2023 IEEE International Conference On Artificial Intelligence Testing (AITest)*. IEEE, 34–39. doi:10.1109/AITest58265.2023.00014

[47] José Miguel Rojas, Mattia Vivanti, Andrea Arcuri, and Gordon Fraser. 2017. A detailed investigation of the effectiveness of whole test suite generation. *Empirical Software Engineering* 22, 2 (2017), 852–893. doi:10.1007/s10664-015-9424-2

[48] Sergio Segura, Gordon Fraser, Ana B Sanchez, and Antonio Ruiz-Cortés. 2016. A survey on metamorphic testing. *IEEE Transactions on software engineering* 42, 9 (2016), 805–824. doi:10.1109/TSE.2016.2532875

[49] Sergio Segura, José A. Parejo, Javier Troya, and Antonio Ruiz-Cortés. 2018. Metamorphic Testing of RESTful Web APIs. *IEEE Transactions on Software Engineering* 44, 11, 1083–1099. doi:10.1109/TSE.2017.2764464

[50] Lijun Shan and Hong Zhu. 2009. Generating structurally complex test cases by data mutation: A case study of testing an automated modelling tool. *Comput. J.* 52, 5 (2009), 571–588. doi:10.1093/comjnl/bxm043

[51] Maximilian Speicher, Brian D. Hall, Ao Yu, Bowen Zhang, Haihua Zhang, Janet Nebeling, and Michael Nebeling. 2018. XD-AR: Challenges and Opportunities in Cross-Device Augmented Reality Application Development. *Proc. ACM Hum.-Comput. Interact.* 2, EICS, Article 7 (jun 2018), 24 pages. doi:10.1145/3229089

[52] Chang-ai Sun, Hui Jin, SiYi Wu, An Fu, ZuoYi Wang, and Wing Kwong Chan. 2024. Identifying metamorphic relations: A data mutation directed approach. *Software: Practice and Experience* 54, 3 (2024), 394–418. arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/spe.3280 doi:10.1002/spe.3280

[53] Christoph Treude. 2024. Qualitative data analysis in software engineering: Techniques and teaching insights. In *Handbook on Teaching Empirical Software Engineering*. Springer, 155–176. doi:10.1007/978-3-031-71769-7_6

[54] Rosalia Tufano, Simone Masiero, Antonio Mastropaolo, Luca Pascarella, Denys Poshyvanyk, and Gabriele Bavota. 2022. Using pre-trained models to boost code review automation. In *Proceedings of the 44th international conference on software engineering*. 2291–2302. doi:10.1145/3510003.3510621

[55] Raja Vavekanand and Kira Sam. 2024. *Llama 3.1: An in-depth analysis of the next-generation large language model*. Technical Report.

[56] Dongwei Xiao, Zhibo Liu, Yuanyuan Yuan, Qi Pang, and Shuai Wang. 2022. Metamorphic testing of deep learning compilers. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 6, 1 (2022), 1–28. doi:10.1145/3508035

[57] Congying Xu, Songqiang Chen, Jiarong Wu, Shing-Chi Cheung, Valerio Terragni, Hengcheng Zhu, and Jialun Cao. 2024. Mr-adopt: Automatic deduction of input transformation function for metamorphic testing. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 557–569. doi:10.1145/3691620.3696020

[58] Jian Yang, Jiajun Zhang, Jiaxi Yang, Ke Jin, Lei Zhang, Qiyao Peng, Ken Deng, Yibo Miao, Tianyu Liu, Zeyu Cui, et al. 2024. Execrepobench: Multi-level executable code completion evaluation. *arXiv preprint arXiv:2412.11990* (2024). doi:10.48550/arXiv.2412.11990

[59] Jie Zhang, Junjie Chen, Dan Hao, Yingfei Xiong, Bing Xie, Lu Zhang, and Hong Mei. 2014. Search-based inference of polynomial metamorphic relations. In *Proceedings of the 29th ACM/IEEE international conference on Automated software engineering*. 701–712. doi:10.1145/2642937.2642994

[60] Zhi Quan Zhou, Shaowen Xiang, and Tsong Yueh Chen. 2015. Metamorphic testing for software quality assessment: A study of search engines. *IEEE Transactions on Software Engineering* 42, 3 (2015), 264–284. doi:10.1109/TSE.2015.2478001

[61] Zhengyang Zhu, Hong-Ning Dai, Hanyang Guo, Zeqin Liao, and Zibin Zheng. 2025. VRExplorer: A Model-based Approach for Semi-Automated Testing of Virtual Reality Scenes. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 482–494. doi:10.1109/ASE63991.2025.00047

Received 2026-03-26; accepted 2026-06-18